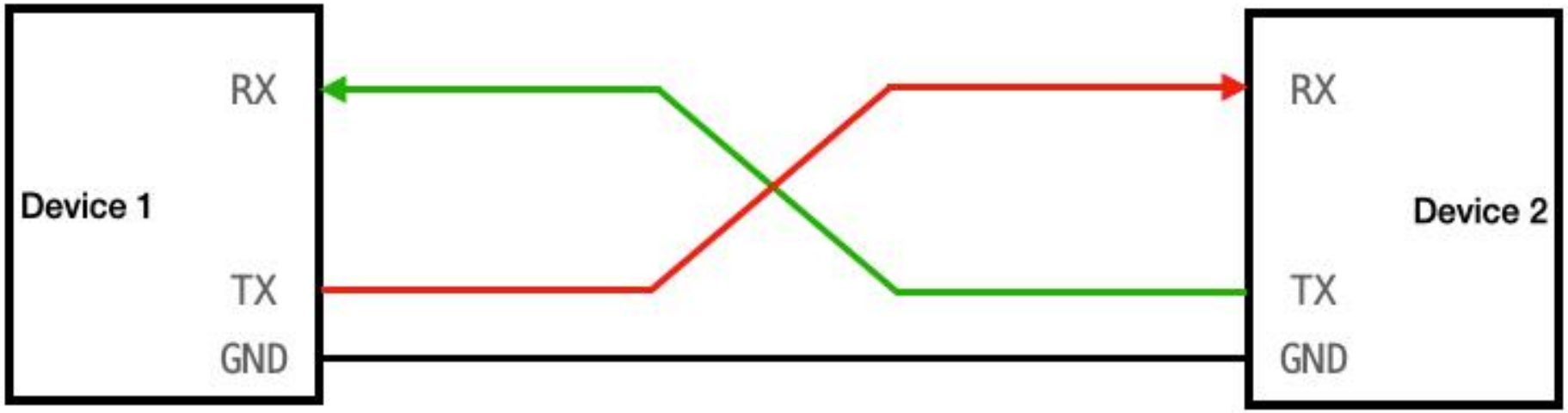
Hardware Description Languages and Verilog -Uart on FPGA

陈刚
教授
人工智能与无人系统研究所
计算机学院
中山大学

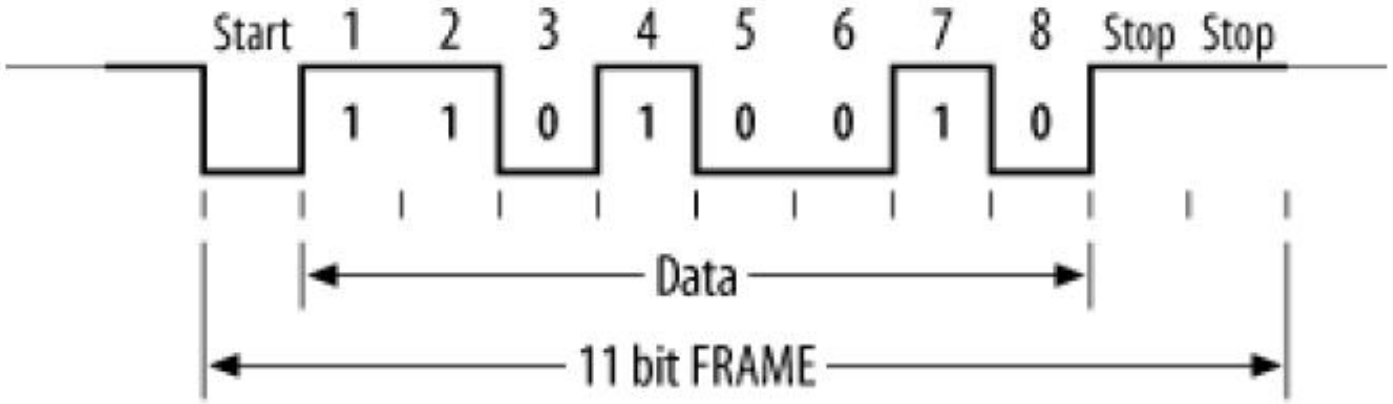


UART串口

- 通用异步接收器/发送器（Universal Asynchronous Receiver/Transmitter），通常称为UART，是一种广泛应用于嵌入式领域的串行、异步、全双工通信协议

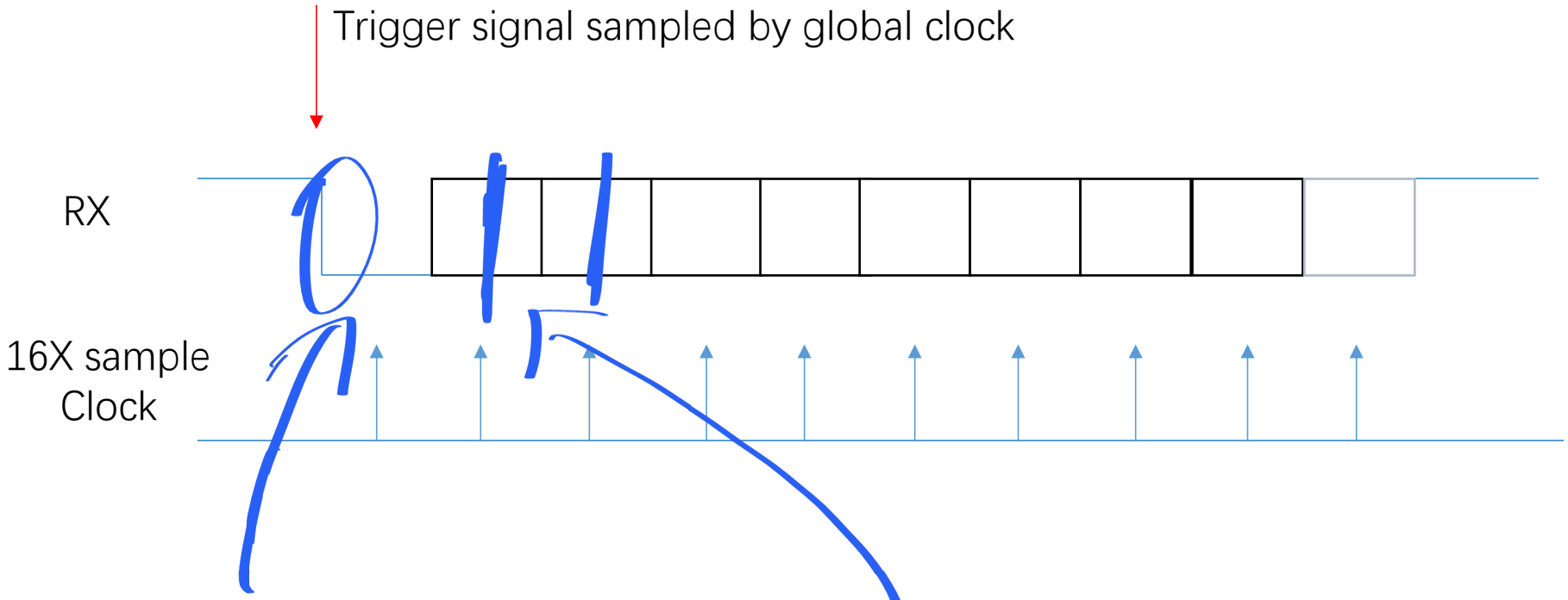


UART串口协议介绍



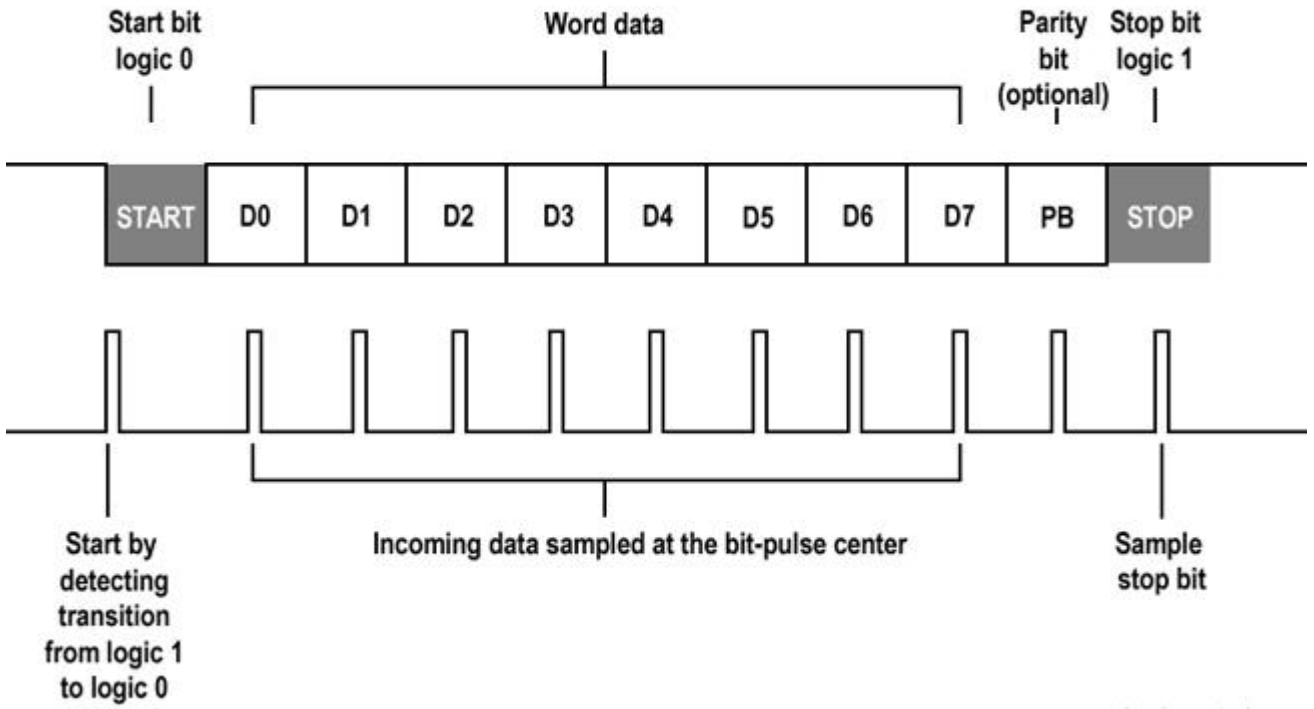
- 快采慢
- 通过下降沿检测起始
- 固定传输速率

UART串口数据采集方法



- 检测到下降沿后开始计数，在该字符发送的中间时刻采集
  - 为什么？在边沿采集容易产生亚稳态

UART串口数据采集方法

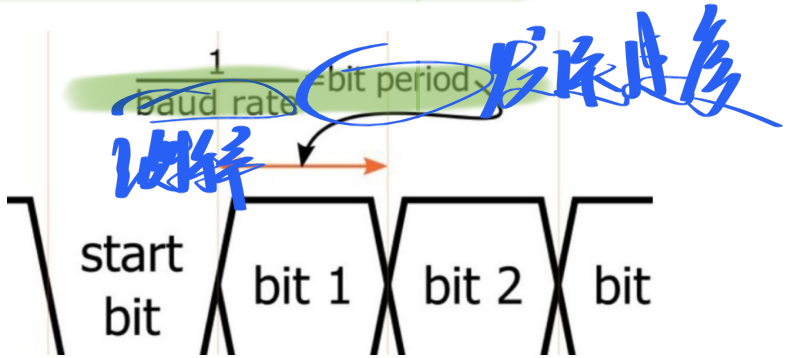


StartBit:每次通信开始时，发送方发送一个逻辑“0”信号，表示传输字符的开始。由于总线空闲时为高电平，因此在开始通信时先发送一个与空闲状态明显不同的信号;

StopBit：它是字符数据的结束标记。它可以是 1 位、1.5 位或 2 位 高电平;

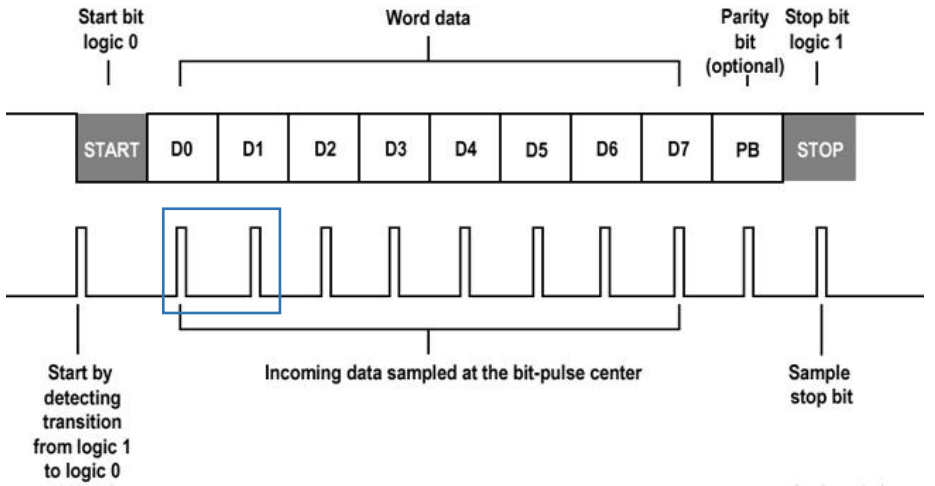
波特率

- 由于异步通信中没有时钟信号，因此两个通信设备需要就波特率达成一致；采样间隔达成一致；
- 波特率用于表示串口发送接收数据位的速度
- 数值上等于一秒钟内发送数据位的个数



- UART协议的一个数据位表示一个单独的0或1，速率由通信双方约定

- 常见波特率：9600，115200



UART回环模块结构

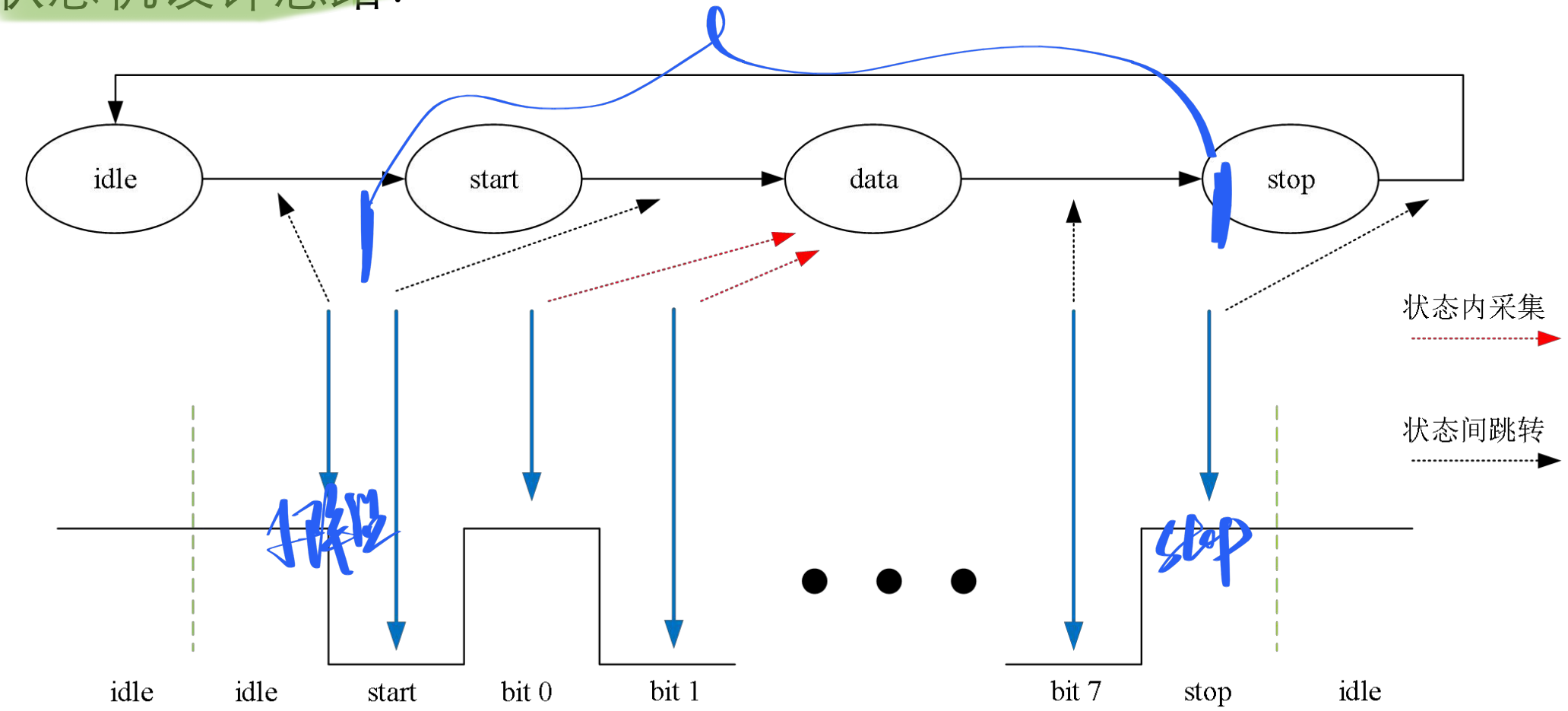


- 功能：从RX接受数据后，直接发送到TX
- 预期结果：

```
1 [2024-10-19 21:55:41.638] SEND >>>>>>>>>>> 12 34 56 78 90 AB CD EF
2 [2024-10-19 21:55:41.650] 12 34 56 78 90 AB CD EF
3
```

UART_RX模块设计

- 状态机设计思路：



UART_RX模块设计

- 快采慢计数器溢出条件计算(设波特率为115200，主时钟50MHz)：
  - 时钟周期 $T = \frac{1}{f} = \frac{1}{50M}\ s$
  - 当前波特率下每 bit 发送时间 $T_{bit} = \frac{1}{baud} = \frac{1}{115200}\ s$
  - 发送 1bit 所需要的周期数 $n = \frac{T_{bit}}{T} = \frac{f}{baud} = \frac{50M}{115200} = 434.027$

UART_RX模块设计

- 计数器控制采集点实现：
  - 在接收到起始信号baudgen_en后开始计数
  - 起始位只计数一半(向右移位实现)
  - 每次计满溢出后清空计数器，并拉高bit_done

- Verilog 代码

```
always @(posedge clk_i or negedge rstn_i) begin
        if (rstn_i == 1'b0) begin
            baud_cnt <= 'h0;
            bit_done <= 1'b0;
    end else if (baudgen_en) begin
            if (!start_bit && (baud_cnt == cfg_div_i)) begin
                baud_cnt <= 'h0;
                bit_done <= 1'b1;
            end else if (start_bit && (baud_cnt == {1'b0, cfg_div_i[15:1]})) begin
                baud_cnt <= 'h0;
                bit_done <= 1'b1;
            end else begin
                baud_cnt <= baud_cnt + 1;
                bit_done <= 1'b0;
            end
    end else begin
            baud_cnt <= 'h0;
            bit_done <= 1'b0;
        end
    end
```

UART_RX模块设计

- RX数据采集：
  - 每次收到bit_done后拉高sampleData，保存数据，将当前数据向右移位
  - 保存8次数据后接收完毕跳转状态
  - 对rx的数据进行三次采样，避免亚稳态，通过后两级采样获取下降沿

- Verilog 代码

```
DATA: begin
    baudgen_en      = 1'b1;
    reg_data_next = {reg_rx_sync[2], reg_data[7:1]};
    if (bit_done) begin
        sampleData = 1'b1;
      if (reg_bit_count == 3'h7) begin
            reg_bit_count_next = 'h0;
            NS = SAVE_DATA;
        end else begin
            reg_bit_count_next = reg_bit_count + 1;
        end
    end
end

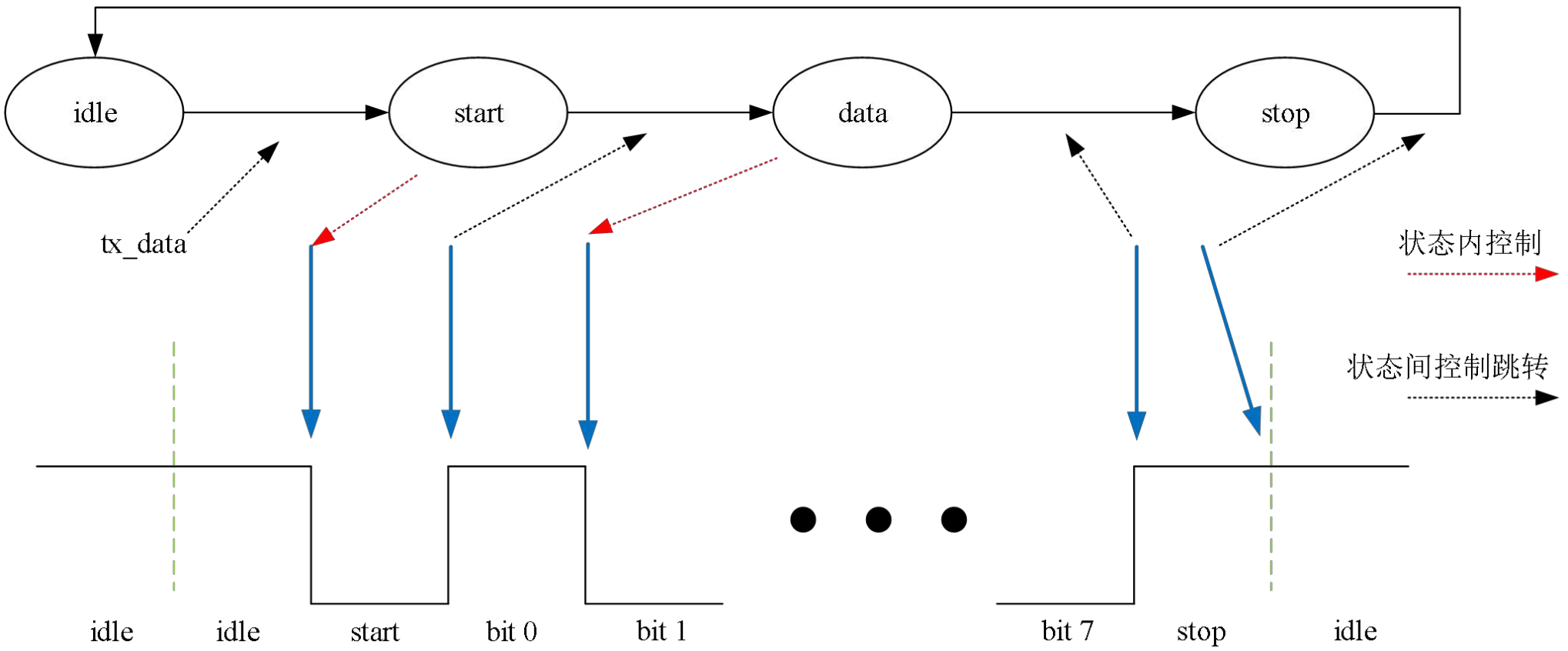
always @(posedge clk_i or negedge rstn_i) begin
    if (rstn_i == 1'b0) begin
        reg_data      <= 8'hff;
        reg_bit_count <= 'h0;
    end else begin
        reg_bit_count <= reg_bit_count_next;
        if (sampleData)
            reg_data <= reg_data_next;
        end
    end

always @(posedge clk_i or negedge rstn_i) begin
    if (rstn_i == 1'b0)
        reg_rx_sync <= 3'b111;
    else
        reg_rx_sync <= {reg_rx_sync[1:0], rx_i};
end

assign s_rx_fall = ~reg_rx_sync[1] & reg_rx_sync[2];
```

UART_TX模块设计

- 状态机设计思路:



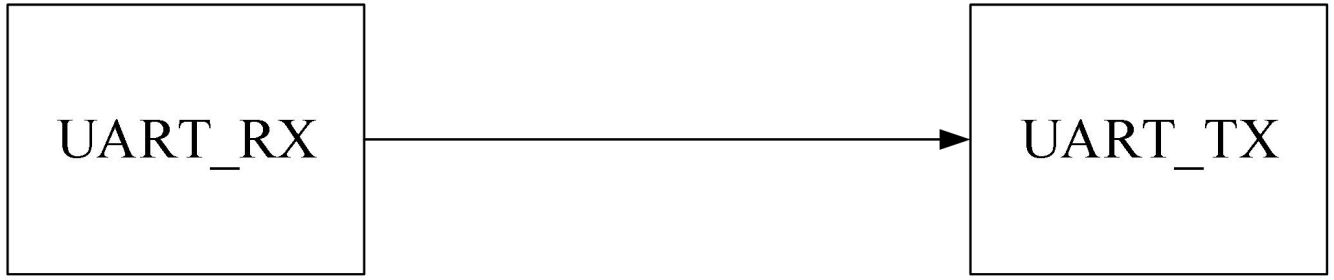
UART_TX模块设计

- TX输出控制：
  - 根据UART协议输出指定的起始位与停止位
  - 每次发送完一位对tx_data进行右移

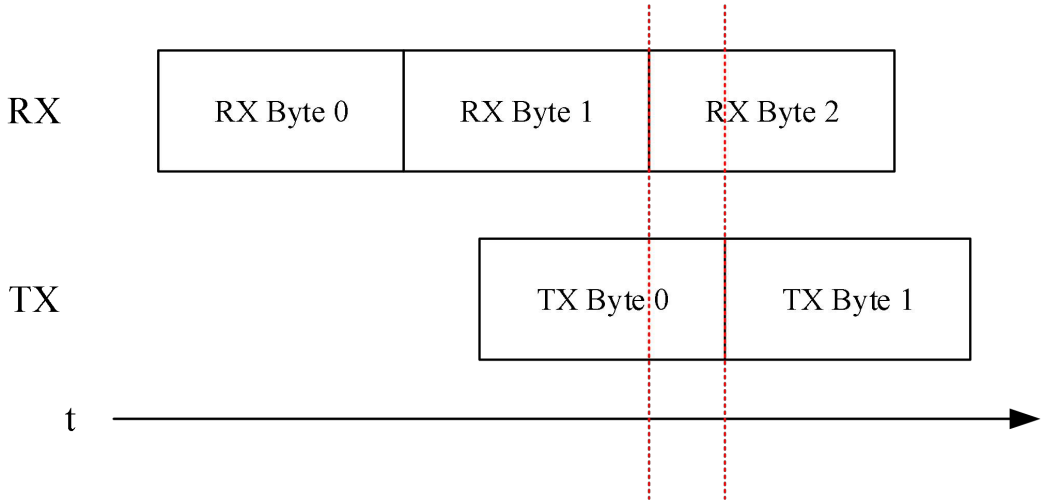
- Verilog 代码

```
START_BIT: begin
    tx_o = 1'b0;
    if (bit_done)
        NS = DATA;
end
DATA: begin
    tx_o = tx_data[0];
    if (bit_done) begin
        if (reg_bit_count == s_target_bits) begin
            reg_bit_count_next = 'h0;
            NS = STOP_BIT_FIRST;
        end else begin
            reg_bit_count_next = reg_bit_count + 1;
            sampleData = 1'b1;
        end
    end
end
STOP_BIT_FIRST: begin
    tx_o       = 1'b1;
    if (bit_done)
        NS = STOP_BIT_LAST;
    else
        NS = IDLE;
end
STOP_BIT_LAST: begin
    tx_o = 1'b1;
    if (bit_done)
        NS = IDLE;
end
```

回环设计

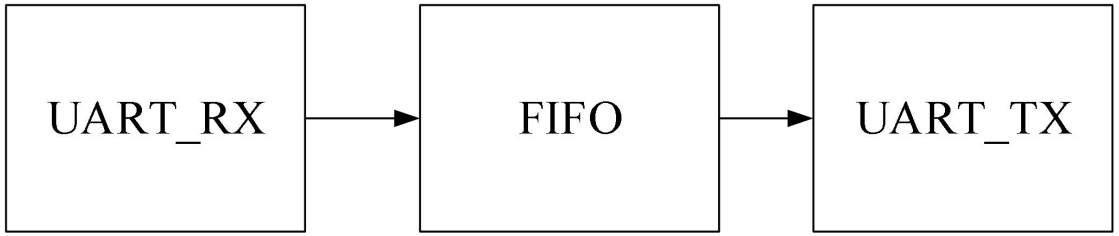


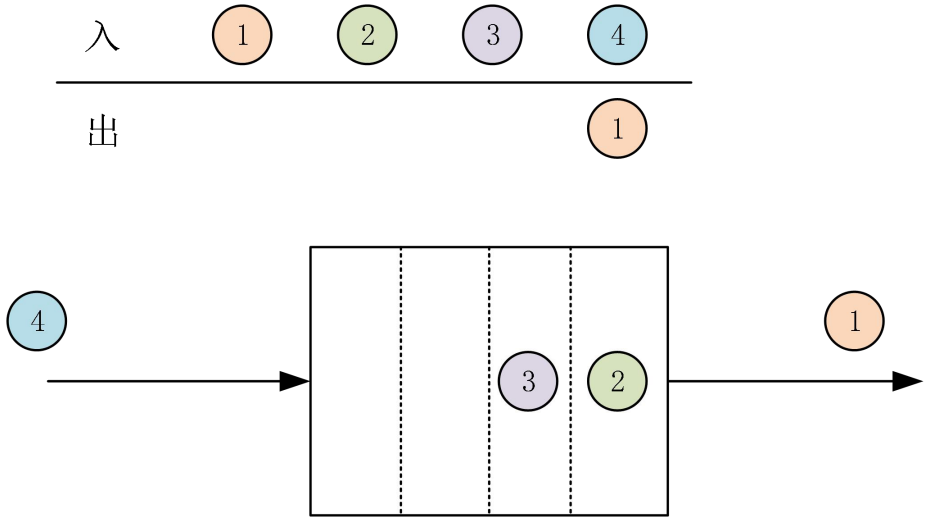
- 考虑模块直接连接
- RX接收到TX启动发送需要时间
- 重叠时间内的数据需要缓存



回环设计

- 流量控制
  - 在路径上增加FIFO缓存数据
  - FIFO类似队列，为先入先出结构
  - 可将接收与发送解耦



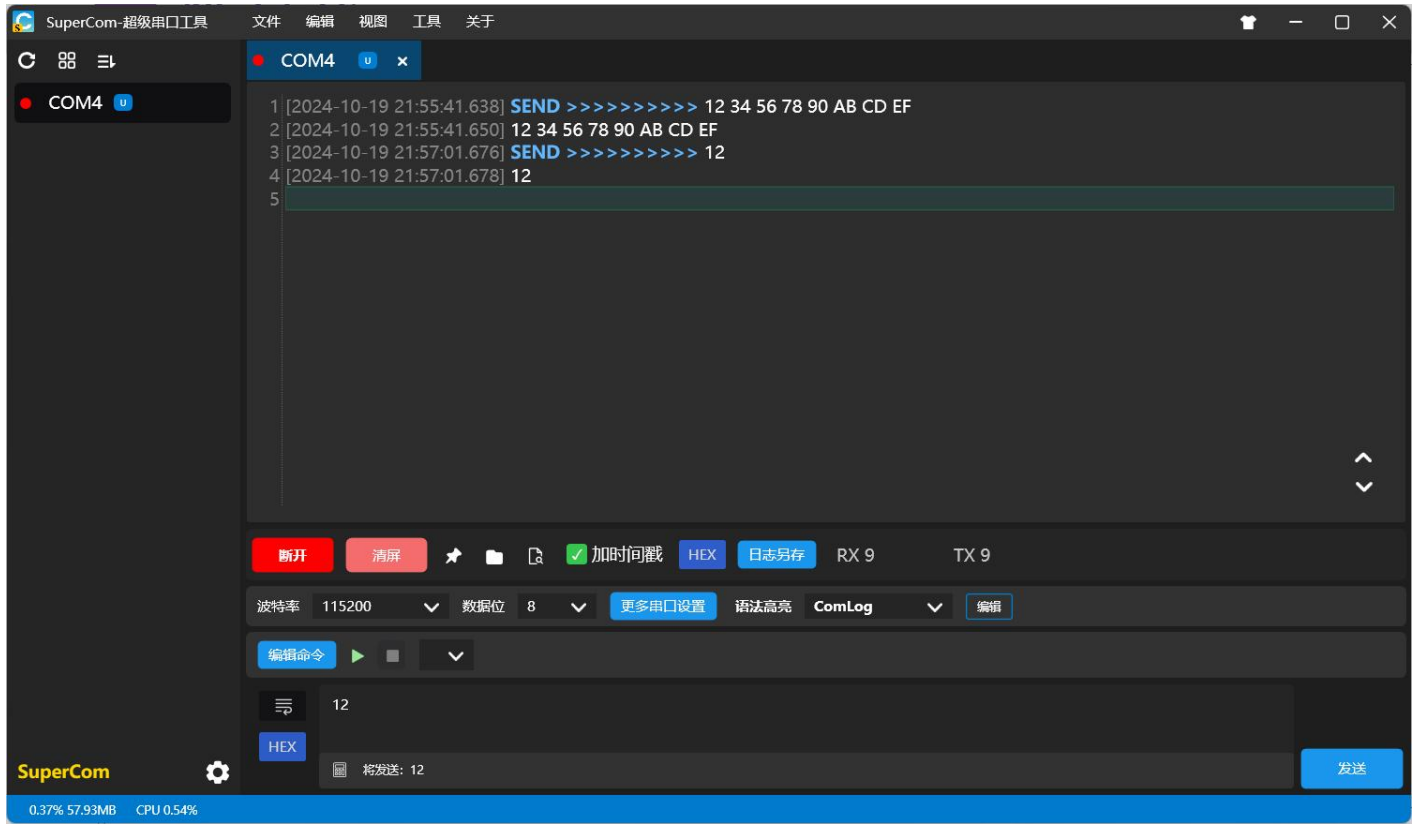


FIFO效果示意图

实验方法

- 烧录代码，在PC机上启动串口调试助手
- 串口调试助手助手有很多版本，自行选择即可
- 如果找不到串口需自行下载安装CH340驱动

- 使用HEX模式收发数据
  - 直接发送字符串会被解析为ASCII码



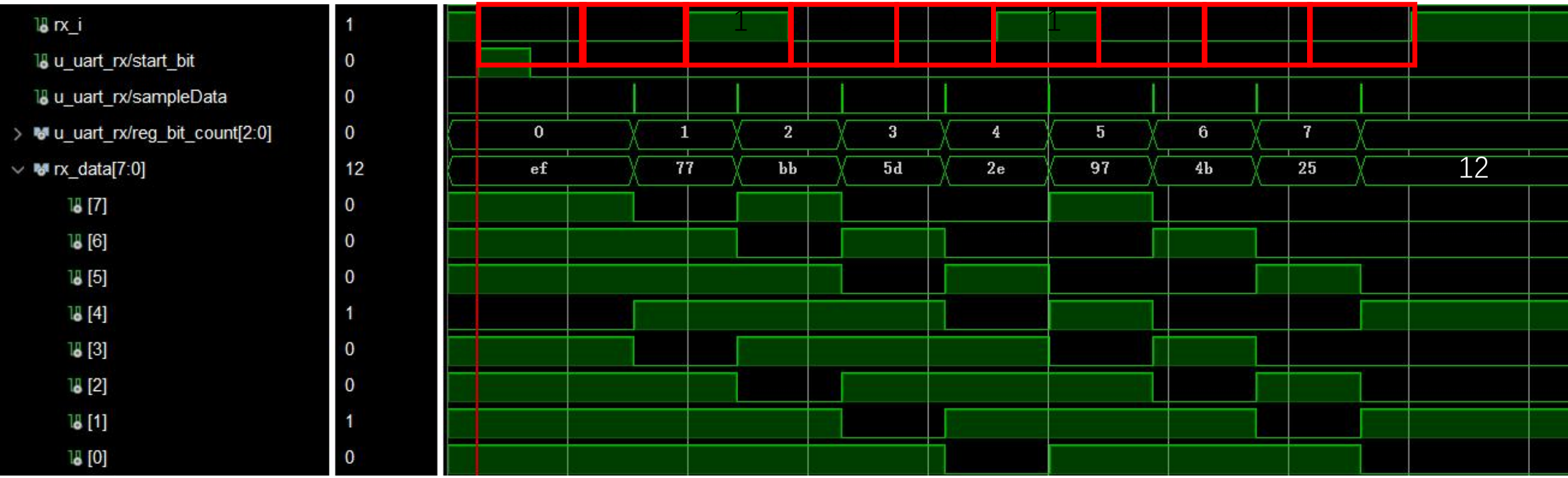
实验效果



12(d) = 0001 0010(b)

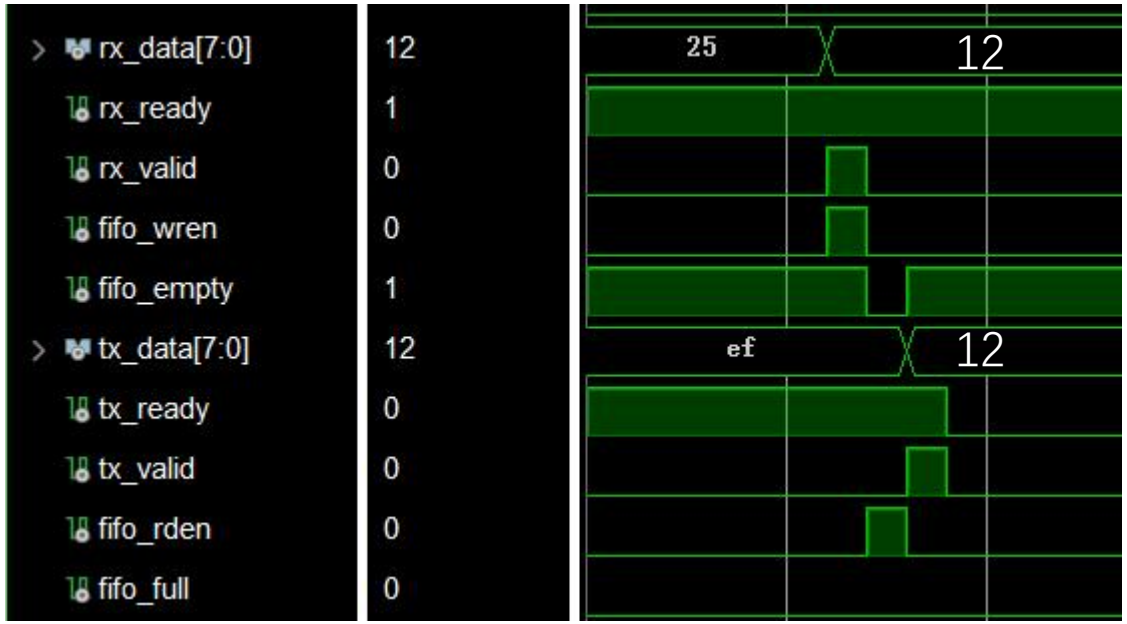
LSB

MSB



实验效果

- 回环部分响应
  - 当端口的valid与ready均为高电平时标识当前数据有效
  - FIFO通过写使能wren将数据写入，通过读使能rden将数据读出
  - FIFO在拉高读使能的下一拍才能输出有效数据，因此需要将tx_valid向后延一拍



实验内容

- 学习UART解析模块代码逻辑
- 根据实验手册中提供的波形图，自行完成回环功能，并使用串口调试助手完成回环功能的测试

实验要求

- 给出完整的回环程序代码
- 验收串口调试助手的回环显示
- 提交完整的实验报告